Merge Sort

We now turn our attention to using a ***divide and conquer*** strategy as a way to improve the performance of sorting algorithms. The first algorithm we will study is the ***merge sort***. ***Merge sort*** is a ***recursive algorithm*** that continually ***splits*** a list in ***half***. If the list is empty or has one item, it is sorted by definition (the base case). If the list has more than one item, we split the list and recursively ***invoke a merge sort*** on ***both halves***. Once the two halves are sorted, the fundamental operation, called a ***merge***, is performed. ***Merging*** is the process of taking two smaller sorted lists and combining them together into a single, sorted, new list: It's highly efficient for large datasets and guarantees ***stable sorting***, meaning it preserves the relative order of ***equal element.***

The algorithm works in two main phases:

- **Divide:** ***Split*** the array into two halves until each subarray contains a single element (which is inherently sorted).
- **Conquer:** ***Merge*** the sorted subarrays back together by comparing elements and placing them in the correct order.

Example: Let's sort the **List** [5, 2, 8, 1, 9] using **Merge Sort.**

Step-by-Step Working

Initial List: **[5, 2, 8, 1, 9]**

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

Step 1: *Divide*

- Split into two halves: [5, 2] and [8, 1, 9].
- Recursively split [5, 2] into [5] and [2].
- Recursively split [8, 1, 9] into [8] and [1, 9], then [1, 9] into [1] and [9].
- Now we have single-element subarrays: [5], [2], [8], [1], [9] (all inherently sorted).

Step 2: *Merge*

- Merge [5] and [2]:
  1) Compare 5 and 2. Take 2 first, then 5.
  2) Result: [2, 5]

- Merge [8] and [1, 9]:
  1) First, merge [1] and [9]: Compare 1 and 9, take 1, then 9.
  2) Result: [1, 9]
  3) Now merge [8] and [1, 9]: Compare 8 with 1 and 9. Take 1, then 8, then 9.
  4) Result: [1, 8, 9]

- Merge [2, 5] and [1, 8, 9]:
  1) Compare elements: 2 vs 1 (take 1), 2 vs 8 (take 2), 5 vs 8 (take 5), then take 8 and 9.
  2) Result: [1, 2, 5, 8, 9]

Final ***Sorted List***: **[1, 2, 5, 8, 9]**

Below, 2 figures show how we can split the data in merge sort and how data will be merged after the comparison of values.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

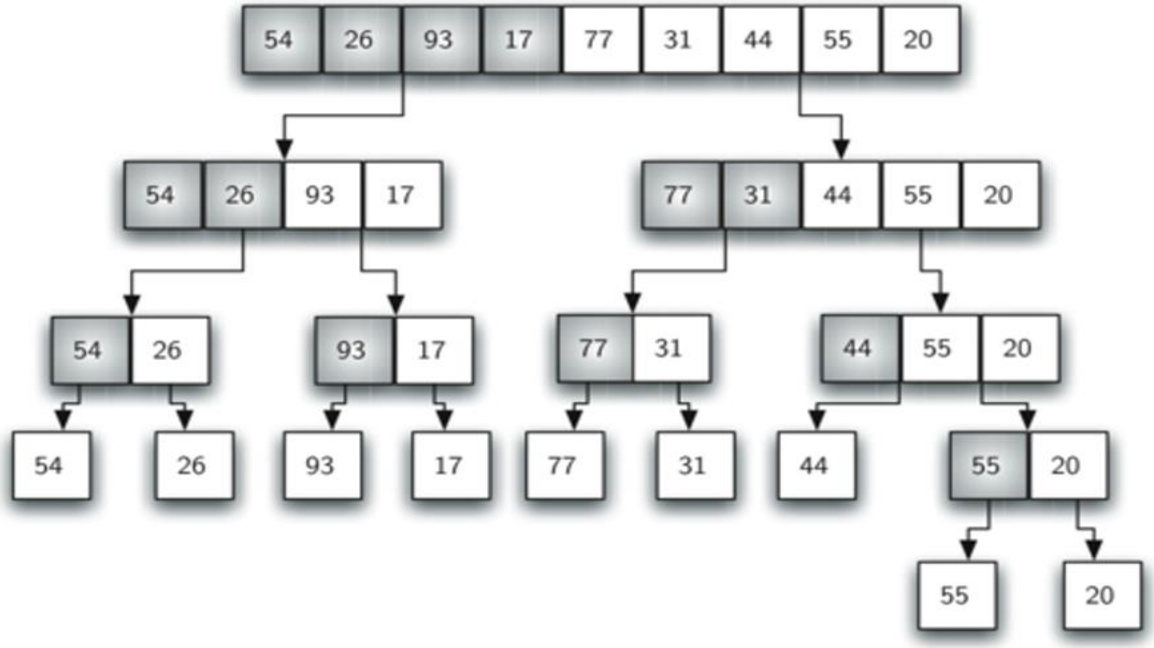

Figure 1: Splitting the List in a Merge Sort

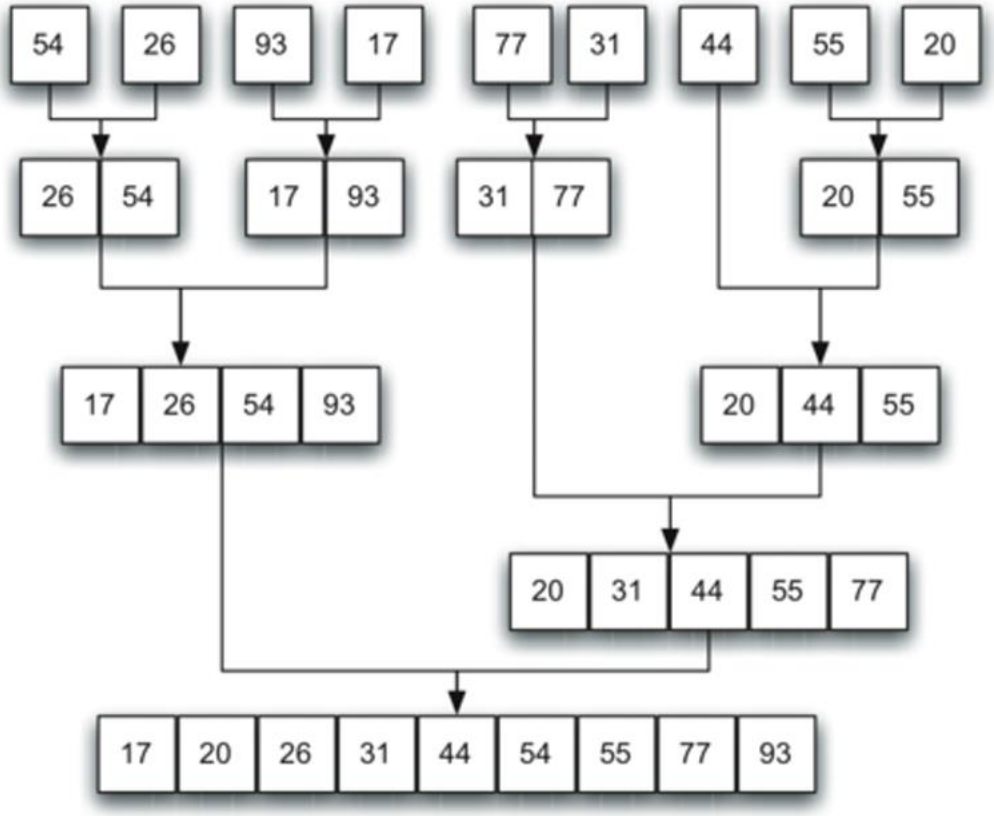

Figure 2: Lists as They Are Merged Together

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

The `**mergeSort()**` function begins by assuming the entire input sequence needs to be sorted. It starts by ***recursively dividing*** the sequence into ***two halves*** until each subarray contains a ***single item***, which is inherently in its proper position. Next, an ***iterative merging*** process is performed over these ***subarrays*** to combine them into ***larger***, ***sorted sequences***. The unordered part of the sequence is initially the whole array, and the process transforms it into a fully ordered sequence through merging. The recursion depth, determined by the size of the array, marks the separation points where the array is split. The inner ***merge operation*** is used to find the ***correct order*** by comparing items from the ***two sorted subarrays*** and placing them into ***a combined, sorted sequence***. Thus, the merge loop starts from the beginning of both subarrays and works its way through, integrating items into their proper positions within the merged result. After completing the merging of all subarrays, the entire sequence is fully sorted.

1- Where do we apply Merge Sort?

Merge Sort is mostly used in scenarios where:

- ***Large Data Sets:*** Its O(n log n) time complexity makes it suitable for sorting large datasets.
- ***External Sorting:*** Sorting data that doesn't fit in memory (e.g., on disk or in databases), as Merge Sort's divide-and-conquer approach works well with sequential access.
- ***Parallel Processing:*** Its divide-and-conquer nature makes it easy to parallelize, suitable for multi-core or distributed systems.
- ***Linked Lists:*** Merge Sort is efficient for sorting linked lists, as it doesn't require random access to elements.
- ***Standard Library Implementations:*** Used in algorithms like Timsort (Python, Java) for its stability and efficiency.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

2- Advantages of Merge Sort

- ***Consistent Performance:*** Guarantees O(n log n) time complexity in all cases (best, average, worst), unlike QuickSort, which can degrade to $O(n^2)$.
- ***Stable Sorting:*** Preserves the relative order of equal elements, making it ideal for sorting objects with multiple attributes.
- ***Predictable:*** Performance doesn't depend on the input data's initial order (unlike Insertion Sort).
- ***Parallelizable:*** The divide-and-conquer approach allows parallel processing, improving performance on multi-core systems.
- ***Efficient for Linked Lists:*** Works well with linked lists, as it doesn't require random access.
- ***External Sorting:*** Well-suited for sorting data stored on disk or in external memory.

3- Disadvantages of Merge Sort

- ***Space Complexity:*** Requires O(n) extra space for the temporary arrays during merging, unlike in-place algorithms like QuickSort or Insertion Sort.
- ***Not In-Place:*** The need for additional memory can be a drawback in memory-constrained environments.
- ***Slower for Small Data Sets:*** The overhead of recursive calls and merging makes it less efficient than Insertion Sort for small arrays (e.g., < 50 elements).
- ***More Complex Implementation:*** Compared to simpler algorithms like Bubble Sort or Insertion Sort, Merge Sort is harder to implement and understand.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

4- Implementation of Merge Sort

1) First Step is to Make ***Function of Merge Sort***.

```
def merge_sort(a_list):
    print("Splitting ", a_list)
    if len(a_list) > 1:
        mid = len(a_list) // 2
        left_half = a_list[:mid]
        right_half = a_list[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0 # Index for left_half, right half, a list

        # Merge the two halves
        while i < len(left_half) and j < len(right_half):
            if left_half[i] <= right_half[j]:
                a_list[k] = left_half[i]
                i += 1
            else:
                a_list[k] = right_half[j]
                j += 1
            k += 1
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

```
    # Copy remaining elements from left_half, if any
    while i < len(left_half):
        a_list[k] = left_half[i]
        i += 1
        k += 1

    # Copy remaining elements from right_half, if any
    while j < len(right_half):
        a_list[k] = right_half[j]
        j += 1
        k += 1

    print("Merging ", a_list)

return a_list
```

2) Call ***Merge Sort Function*** for Data

```
# Test the function
a_list = [54, 26, 93, 17, 77, 31, 44, 55, 20]
merge_sort(a_list)
print("Final sorted list:", a_list)
```

Use the above ***merge sort technique*** for ***stacks*** and ***queues*** as previously applied to ***bubble, selection, and insertion sorts.***

Hint: For the ***sorting*** of the ***stack*** and ***queue***, just ***integrate*** merge sort function in stack and queue.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

5- Practice Questions

Below are the ***3 practice questions*** based on the ***Merge Sort*** implementation that will help you to learn and practice ***Merge Sort*** effectively with respect to Stack and Queue as well:

1. Prioritizing Tasks in a Project Management System

A project manager uses a ***queue*** to track incoming tasks by ***priority levels*** (10 = highest priority, 1 = lowest), such as: [7, 3, 9, 2, 5, 8] (enqueued in order of submission).

- ***Dequeue*** all tasks into a list.
- Use **Merge Sort** to arrange tasks in ***descending*** *and* ***Ascending order*** of priority (highest priority first).
- ***Enqueue*** the sorted list back into the queue.
- ***Print*** the queue contents by ***dequeuing*** all elements at the end.

2. Organizing Inventory in a Warehouse

A warehouse stores ***product IDs*** in a ***stack*** in the order they were received (e.g., IDs [205, 201, 210, 202, 208]).

- ***Pop*** all IDs into a list.
- Apply ***Merge Sort*** to sort IDs in ***Descending*** and ***Ascending*** order.
- ***Push*** the ***sorted IDs*** back into a new stack.
- Finally, ***pop all IDs*** from the stack and display them.

3. Airport Baggage Handling System

An airport manages baggage using two data structures for efficiency:

- ***Incoming Baggage (Queue):*** Bags arriving from flights are enqueued by tag numbers in the order they are unloaded.
- ***Claimed Baggage (Stack):*** Bags claimed by passengers are pushed onto a stack for temporary holding.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

The airport staff wants to sort all tag numbers during a system reorganization.

Step 1: Build the Data Structures

- Implement a Queue class with the following functions:

- enqueue(tag_id) → add tag at the rear.
- dequeue() → remove tag from the front.
- peek() → get the first tag without removing it.
- is_empty() and is_full() → check status.
- display() → print the current queue.

- Implement a Stack class with the following functions:

- push(tag_id) → add tag on top.
- pop() → remove tag from the top.
- peek() → get the top tag without removing it.
- is_empty() and is_full() → check status.
- display() → print the current stack.

Step 2: Perform Operations

- Dequeue all incoming bags into a list.
- Pop all claimed bags into another list.
- Merge both lists into a single list.
- Apply Merge Sort to arrange all tag IDs in ascending order.
- Reinsert the sorted IDs:
- First half back into the Queue (incoming section).
- Second half back into the Stack (claimed section).

Step 3: Output Requirements

- Print the initial state of both Queue and Stack.
- Show the merged list before sorting.
- Show the sorted list after applying Merge Sort.
- Print the reinserted Queue and Stack.
- Finally, display all bags again by:
- Dequeuing from the Queue.
- Popping from the Stack.

Hard work beats talent when talent doesn't work hard.
Tim Notke

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk